

NATIONAL ENGINEERING CENTER
University of the Philippines

6.0

Stateful Multi-Agent Systems

Eugene Rex L. Jalao PhD

Professor

Department of Industrial Engineering and Operations Research

University of the Philippines Diliman

eljalao@up.edu.ph

UP NEC

Module Outline

- Introduction to Agentic AI
  - Capstone Project Case Proposal Generation
- Deep Learning, Transformers and LLMs
  - Capstone Project Business Case Workflow
- Agentic Workflows
  - Case Study and Capstone Project Work
- Agentic RAGs
  - Case Study and Capstone Project Work
- Orchestration
  - Case Study and Capstone Project Work
- **Stateful Multi-Agent Systems**
  - **Case Study and Capstone Project Work**
- Evaluation
  - Case Study and Capstone Project Work
- LLMOps
  - Capstone Project Presentation

ERLJalao Copyright UPNEC

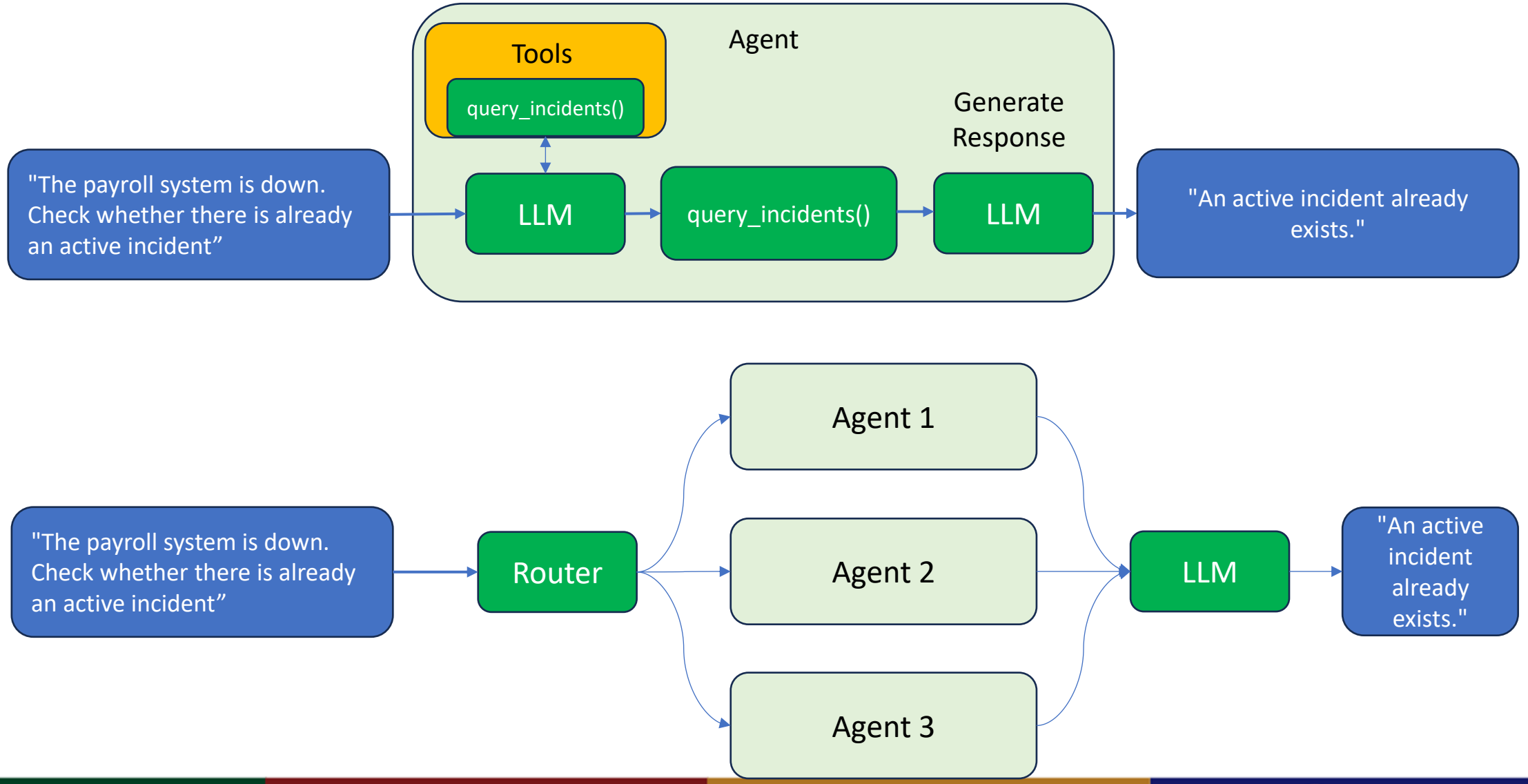
Outline for this Session

- Introduction to Multi-Agent Systems
- Designing Agent Roles and Responsibilities
- Multi-Agent Architecture Patterns
- Agent Communication, Handoffs, and Shared State
- Stateful Multi-Agent Workflows
- Supervisor, Routing, and Workflow Control
- Human-in-the-Loop Multi-Agent Systems
- Project Case Study

From a Single Agent to Multiple Agents

- **Definition:** A **multi-agent system** uses several agents that perform different roles within the same overall task.
- Each agent may have its own instructions, tools, knowledge, context, and responsibilities.
- Agents exchange information through controlled handoffs or shared workflow state.
- An orchestrator or supervisor may coordinate which agent acts next.
- The objective is not to add more LLMs, but to divide complex work into manageable responsibilities.

Single Agents vs Multiple Agents



An AI Agent in a Multi-Agent System

- Each agent has a **specific responsibility** within the larger workflow.
- An agent observes its available context and decides what action to propose.
- The application controls which actions the agent is permitted to perform.
- Different agents can have access to different tools or information.
- Agent specialization reduces the amount of responsibility assigned to one model.
- A multi-agent system is therefore not simply several chatbots talking to each other. It is a **structured workflow of bounded AI workers**.

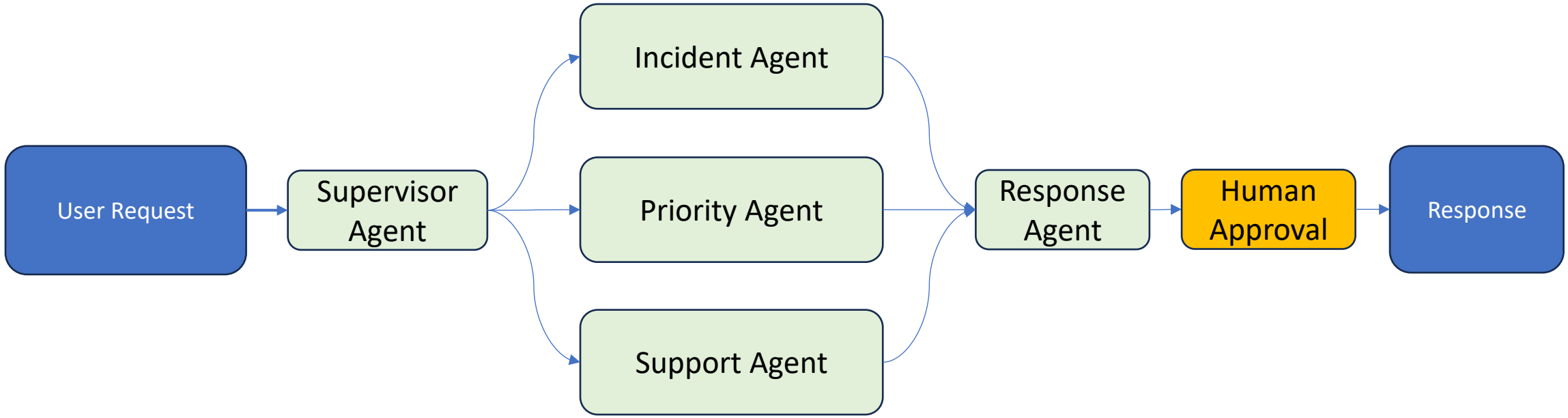
Why Use Multiple Agents?

- Complex business processes often contain tasks requiring different knowledge and capabilities.
- Specialized agents can focus on narrower responsibilities.
- Different agents can use different tools, databases, RAG collections, or models.
- Responsibilities can be separated between analysis, verification, decision support, and action.
- Agent outputs can be reviewed by another agent or by a human before execution.
- Large workflows can become easier to understand when responsibilities are explicitly separated.

Single Agent vs Multi-Agent System

Single Agent	Multi-Agent System
One agent performs several responsibilities.	Responsibilities are distributed among agents.
The same context may contain everything.	Each agent may receive only relevant context.
One agent chooses among many tools.	Different agents may own different tools.
Workflow logic is relatively simple.	Coordination and handoffs become important.
Easier to develop and debug.	Better suited to clearly separable complex tasks.
Lower coordination overhead.	Requires state, routing, and workflow control.

Example: Operations Incident Management



Multi-Agent Systems Require Coordination

- Adding agents creates new questions:
  - Which agent should act first?
  - Which agent should act next?
  - What information should be passed between agents?
  - Where should intermediate results be stored?
  - What happens when an agent fails?
  - What happens when two agents disagree?
  - When should the workflow stop?
  - Which actions require human approval?

When Should We Use a Multi-Agent System?

- **Multi-agent architecture is useful when:**
  - The workflow contains clearly separable responsibilities.
  - Different steps require different tools or knowledge.
  - Different permissions are needed for different actions.
  - Independent verification is valuable.
  - Work can be performed sequentially or in parallel.
  - Human review must occur between stages.
- **A single agent may still be preferable when:**
  - The task is simple and short.
  - One agent can safely access all required tools.
  - The workflow contains few decision points.
  - Additional agents would only increase complexity.

LangGraph



- **LangGraph is a framework for orchestrating stateful AI and agent workflows as graphs.**
  - A workflow is represented as a collection of **nodes connected by edges**.
  - Nodes perform work such as calling an LLM, executing a tool, validating output, or requesting human input.
  - Edges determine how execution moves between nodes.
  - State carries information through the workflow.
  - Conditional edges allow the workflow to branch dynamically.
  - Checkpoints allow workflows to pause and resume.
  - Human decisions can be inserted into the execution flow.
  - Deterministic Python logic and LLM-driven decisions can coexist in the same graph.

Introduction to the Northwind Database

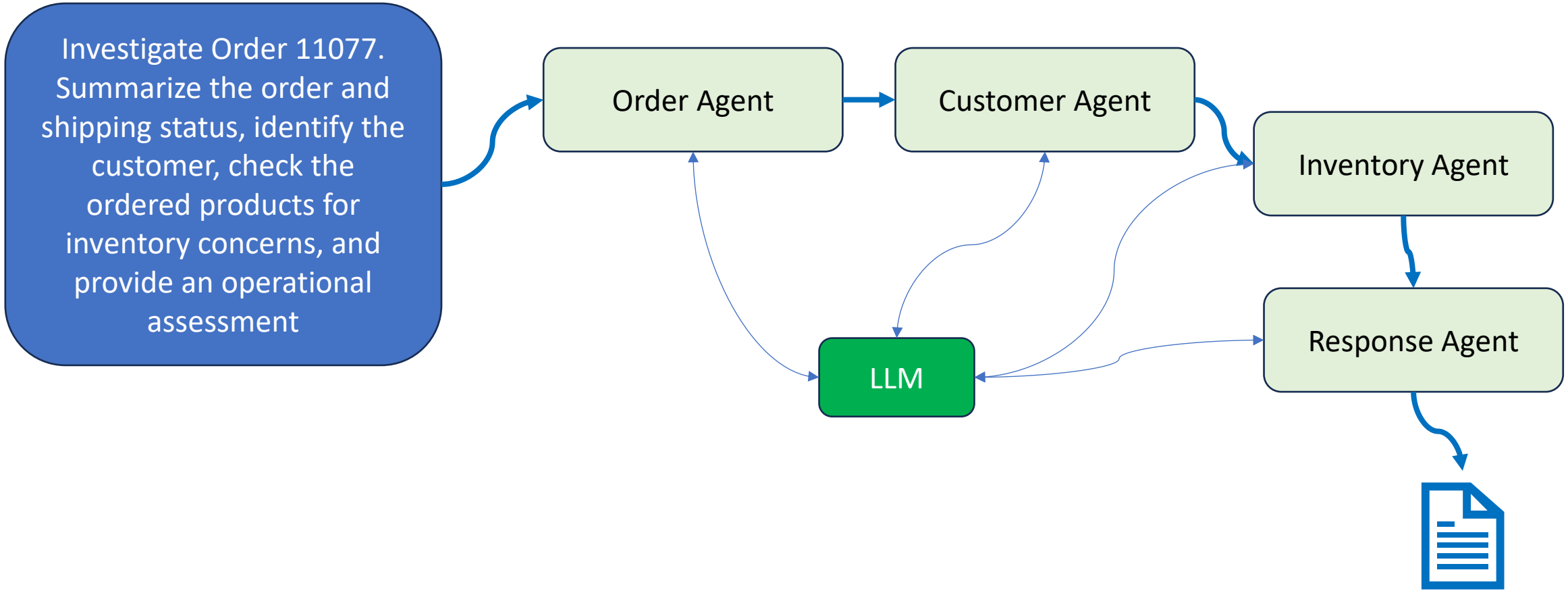
- **Northwind** is a classic sample relational database originally created by Microsoft to demonstrate database concepts and business applications.
  - It represents a fictional trading company that sells food and beverage products to customers worldwide.
  - The database contains interconnected business data such as **customers, orders, products, suppliers, employees, and shippers**.
  - Its relational structure makes it useful for practicing queries, analytics, workflow automation, and agent-based applications.
  - Common tables include Customers, Orders, Order Details, Products, Suppliers, Employees, and Shippers.
  - Relationships between tables allow agents to answer multi-step business questions such as order status, customer history, and product availability.

Introduction to Multi-Agent Systems

- **Exercise 1: Sequential Multi-Agent Workflow**
- **Objectives**
  - Build a basic multi-agent workflow using LangGraph.
  - Assign specialized responsibilities to Order, Customer, Inventory, and Response agents.
  - Query northwind.db as the factual source for agent decisions.
  - Observe the execution sequence using a LangGraph visualization.
- **Business Case**
  - A Northwind operations manager wants to investigate an order without manually checking several database tables.

Introduction to Multi-Agent Systems

- **Exercise 1: Sequential Multi-Agent Workflow**



Introduction to Multi-Agent Systems

- **Exercise 1: Sequential Multi-Agent Workflow**

```python
#%% EXERCISE 1: INTRODUCTION TO MULTI-AGENT SYSTEMS
# NORTHWIND ORDER INVESTIGATION

import json
import sqlite3
import ollama
from typing import TypedDict
from langgraph.graph import StateGraph,START,END

DB="northwind.db"
MODEL="qwen3.5:4b"

# ============================================================
# COMMON HELPERS
# ============================================================

def ask_json(system_prompt,user_prompt):
    r=ollama.chat(
        model=MODEL,
        messages=[
            {"role":"system","content":system_prompt},
            {"role":"user","content":user_prompt}
        ],
        format="json",
        think=False
    )
    return json.loads(r.message.content)

def add_trace(state,message):
    return state.get("trace",[])+[message]
```

```
ORDER AGENT
Order #11077 from customer RATTC was placed on May 6, 2021, with a required delivery
date of June 3, 2021. The freight charge is $28. Shipped by United Package to the USA,
the order has not been shipped as the shipped date remains null.

CUSTOMER AGENT
The customer Rattlesnake Canyon Grocery, located in Albuquerque, USA and contacted by
Paula Wilson, has a total of 174 orders in the database.

INVENTORY AGENT
Low stock concerns detected for: Aniseed Syrup (ProductID:3), Mascarpone Fabioli
(ProductID:32), Wimmers gute Semmelknödel (ProductID:64), and Louisiana Hot Spiced Okra
(ProductID:66). Potential shortage alert issued for Chang (ProductID:2).

FINAL RESPONSE
Order #11077 from RATTC (Rattlesnake Canyon Grocery) placed on May 6, 2021, with a
required delivery date of June 3, 2021, remains unshipped. Operational priority includes
restocking low items: Aniseed Syrup, Mascarpone Fabioli, Wimmers gute Semmelknödel, and
Louisiana Hot Spiced Okra, plus monitoring for a potential shortage of Chang.

EXECUTION TRACE
Order Agent [LLM] -> Customer Agent [LLM] -> Inventory Agent [LLM] -> Response Agent
[LLM]
```

Outline for this Session

- Introduction to Multi-Agent Systems
- **Designing Agent Roles and Responsibilities**
- Multi-Agent Architecture Patterns
- Agent Communication, Handoffs, and Shared State
- Stateful Multi-Agent Workflows
- Supervisor, Routing, and Workflow Control
- Human-in-the-Loop Multi-Agent Systems
- Project Case Study

Agent Roles

- A multi-agent system works best when each agent has a clearly defined responsibility.
- Poorly defined roles can cause duplicated work, conflicting decisions, and unnecessary agent communication.
- Each agent should own a specific part of the business process.
- Agents should receive only the tools, knowledge, and permissions needed for their assigned role.
- Clear roles also make the workflow easier to test, monitor, and audit.

Business Workflow

- Before creating agents, first decompose the business process.
- Then decide which steps require separate agents.

Business Request
↓
Identify Incident
↓
Assess Priority
↓
Identify Responsible Team
↓
Prepare Response
↓
Approve Response
↓
Send Response

Identify Incident → Incident Agent
Assess Priority → Priority Agent
Identify Support Team → Support Agent
Prepare Response → Response Agent
Coordinate Workflow → Supervisor Agent
Approve Response → Human

Anatomy of an Agent Role

Element	Question to Define
Role	What responsibility does this agent own?
Goal	What result should the agent produce?
Inputs	What information can the agent receive?
Knowledge	What RAG sources or context can it access?
Tools	What functions or systems can it use?
Output	What structured result must it return?
Authority	What actions is it allowed to perform?
Escalation	When should it stop and request help?

Bounded Authority

- Agents should have **only the authority required to perform their roles**.
- Separate recommendation, approval, and execution whenever the action is consequential.

Incident Agent

Allowed:

✓ Search incident records

✓ Retrieve incident details

Not Allowed:

✗ Change incident priority

✗ Send customer messages

Response Agent

Allowed:

✓ Draft a response

Not Allowed:

✗ Approve the response

✗ Send the response

Supervisor Agent

Allowed:

✓ Route tasks

✓ Validate outputs

✓ Update workflow state

Not Allowed:

✗ Override required human approval

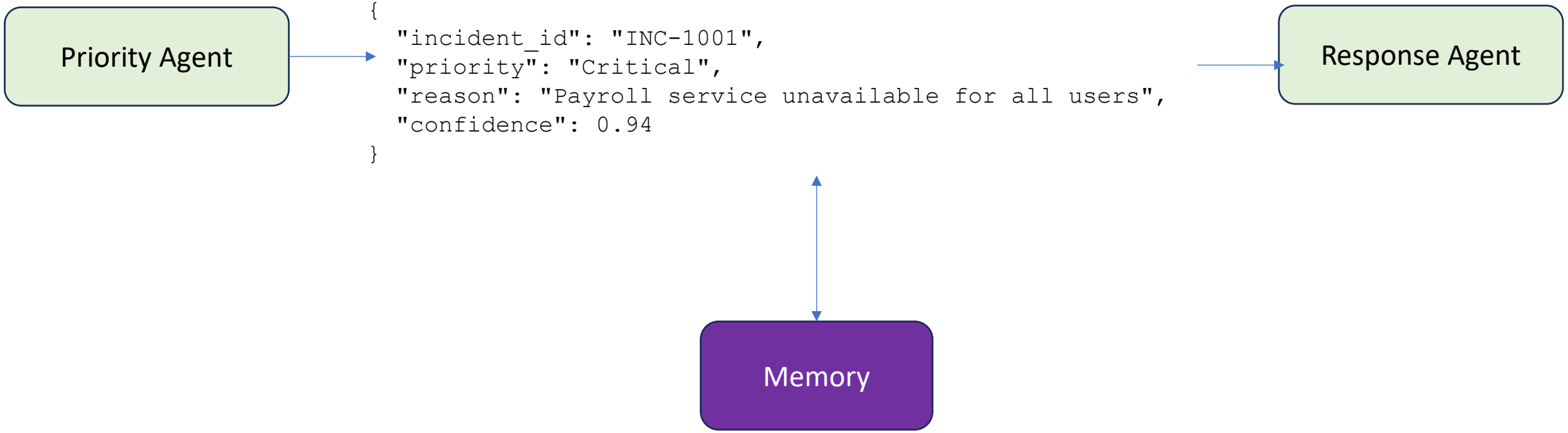
Avoid Overlapping Agent Responsibilities

- A common design problem is giving several agents the same responsibility.
- If two agents perform substantially the same task, ask whether both are necessary.

Agent A → Determine priority
Agent B → Check priority
Agent C → Decide priority
Supervisor → Decide priority again

Define Agent Inputs and Outputs as Contracts

- Agents should communicate using **structured outputs**, not unrestricted conversational text whenever possible.



Designing the Incident-Management Agent Team

Agent	Primary Responsibility	Tools / Knowledge	Output
Supervisor Agent	Coordinate workflow and select the next step.	Workflow state and routing rules.	Next agent/action.
Incident Agent	Retrieve the relevant incident.	query_incidents()	Incident record.
Priority Agent	Assess incident priority.	Policy + calculate_priority()	Priority assessment.
Support Agent	Determine the responsible team or delegate.	lookup_delegate()	Assigned team/person.
Response Agent	Prepare a human-readable response.	Incident context and templates.	Draft response.
Human Reviewer	Approve, reject, or request correction.	Complete workflow context.	Approval decision.

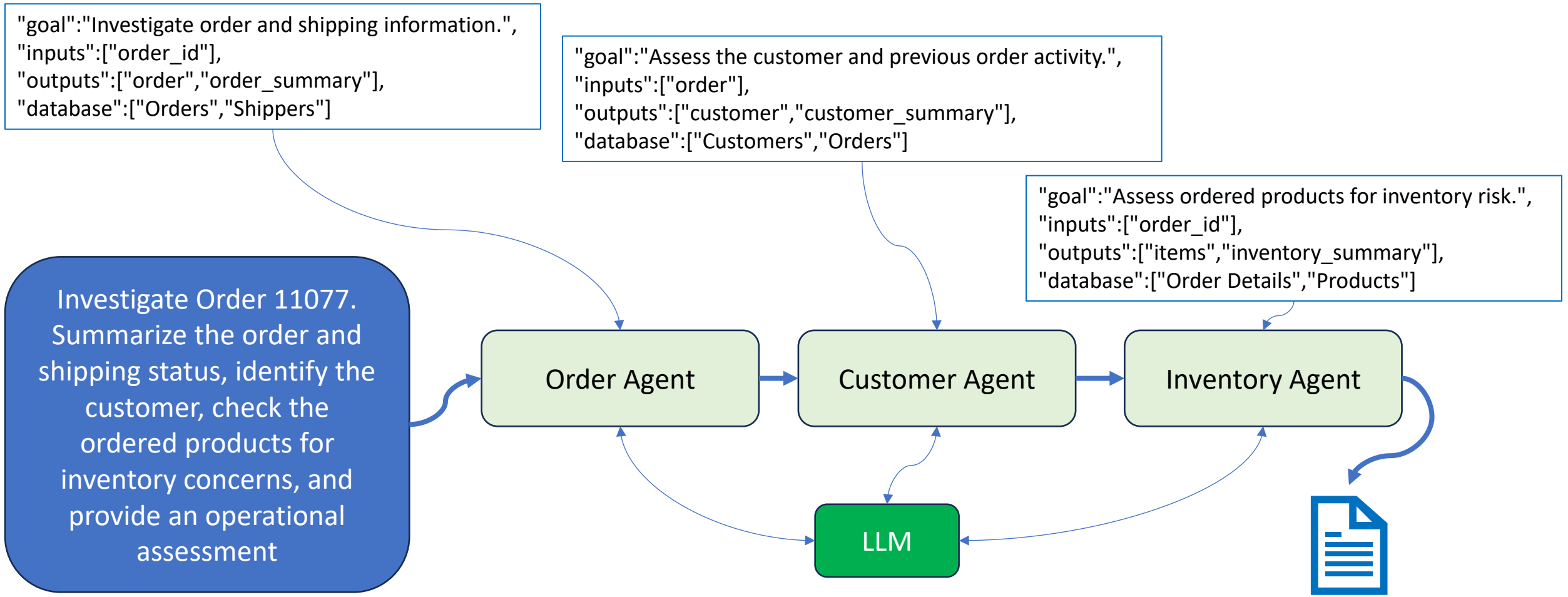
Designing Agent Roles and Responsibilities

- **Exercise 2: Give Every Agent a Bounded Role**
- **Objectives**
  - Define clear responsibilities for each agent in the Northwind order-investigation workflow.
  - Restrict each agent to only the business data required for its assigned role.
  - Define explicit inputs and outputs for agent-to-agent communication
  - Demonstrate how bounded roles reduce overlapping responsibilities and uncontrolled agent behavior.
- **Business Case**
  - The workflow is expanded into three specialized responsibilities.

Agent	Responsibility	Database Access
Order Agent	Investigate order and shipping status	Orders, Shippers
Customer Agent	Assess the customer and order history	Customers, Orders
Inventory Agent	Assess product and inventory risk	Order Details, Products

Designing Agent Roles and Responsibilities

- **Exercise 2: Give Every Agent a Bounded Role**



Designing Agent Roles and Responsibilities

- **Exercise 2: Give Every Agent a Bounded Role**

```
#%% EXERCISE 2: DESIGNING AGENT ROLES AND RESPONSIBILITIES
# ============================================================
# ROLE DEFINITIONS
# ============================================================

role_specs={
    "order_agent":{
        "goal":"Investigate order and shipping information.",
        "inputs":["order_id"],
        "outputs":["order","order_summary"],
        "database":["Orders","Shippers"]
    },
    "customer_agent":{
        "goal":"Assess the customer and previous order activity.",
        "inputs":["order"],
        "outputs":["customer","customer_summary"],
        "database":["Customers","Orders"]
    },
    "inventory_agent":{
        "goal":"Assess ordered products for inventory risk.",
        "inputs":["order_id"],
        "outputs":["items","inventory_summary"],
        "database":["Order Details","Products"]
    }
}

# ============================================================
```

```
AGENT ROLE CONTRACTS

order_agent
 Goal: Investigate order and shipping information.
 Inputs: order_id
 Outputs: order, order_summary
 Database: Orders, Shippers

customer_agent
 Goal: Assess the customer and previous order activity.
 Inputs: order
 Outputs: customer, customer_summary
 Database: Customers, Orders

inventory_agent
 Goal: Assess ordered products for inventory risk.
 Inputs: order_id
 Outputs: items, inventory_summary
 Database: Order Details, Products

ORDER AGENT
Order 11077 from customer RATTC was placed on 2021-05-06 with a
of 2021-06-03. The order has not been shipped (ShippedDate: nul
'United Package' shipper in USA with a freight cost of 28
```

Outline for this Session

- Introduction to Multi-Agent Systems
- Designing Agent Roles and Responsibilities
- **Multi-Agent Architecture Patterns**
- Agent Communication, Handoffs, and Shared State
- Stateful Multi-Agent Workflows
- Supervisor, Routing, and Workflow Control
- Human-in-the-Loop Multi-Agent Systems
- Project Case Study

Multi-Agent Architecture Patterns

- **Definition:** A **multi-agent architecture pattern** describes how multiple agents are organized, how tasks move between them, and how decisions are coordinated.
- Different workflows require different coordination structures.
  - Sequential Pipeline
  - Router–Specialist
  - Supervisor–Specialist
  - Delegation / Handoff
  - Parallel Agents with Aggregation
  - Reviewer–Worker
  - Hierarchical Multi-Agent Architecture

Overview

Pattern	Function	Best Fit
Sequential Pipeline	Agents execute in a predetermined order.	Stable step-by-step processes.
Router–Specialist	A router selects the appropriate specialist.	Requests belonging to different categories.
Supervisor–Specialist	A supervisor dynamically coordinates several specialists.	Complex workflows requiring multiple agents.
Delegation / Handoff	One agent transfers responsibility to another.	Tasks requiring changing expertise.
Parallel + Aggregation	Several agents work simultaneously and results are combined.	Independent analyses or subtasks.
Reviewer–Worker	One agent creates output and another checks it.	Quality-sensitive workflows.
Hierarchical	Supervisors manage groups of specialist agents.	Large and complex workflows.

Sequential Multi-Agent Pattern

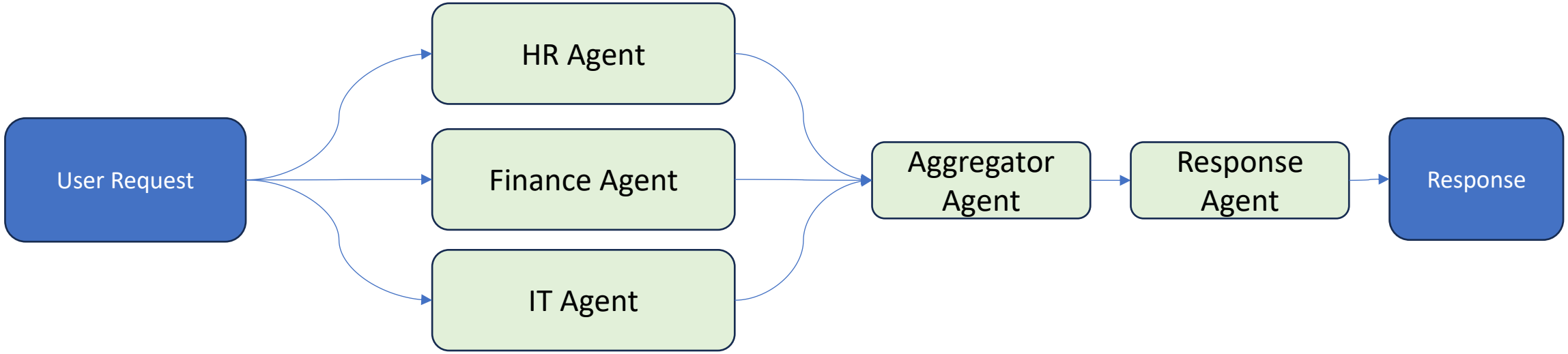
- Agents execute one after another in a predefined sequence.



- **Use this pattern when:**
  - The business process already has a clear sequence.
  - Each stage depends on information produced by the previous stage.
  - Workflow transitions are predictable.
  - Auditability and traceability are important.

Parallel Agents with Aggregation

- Several agents work independently at the same time.



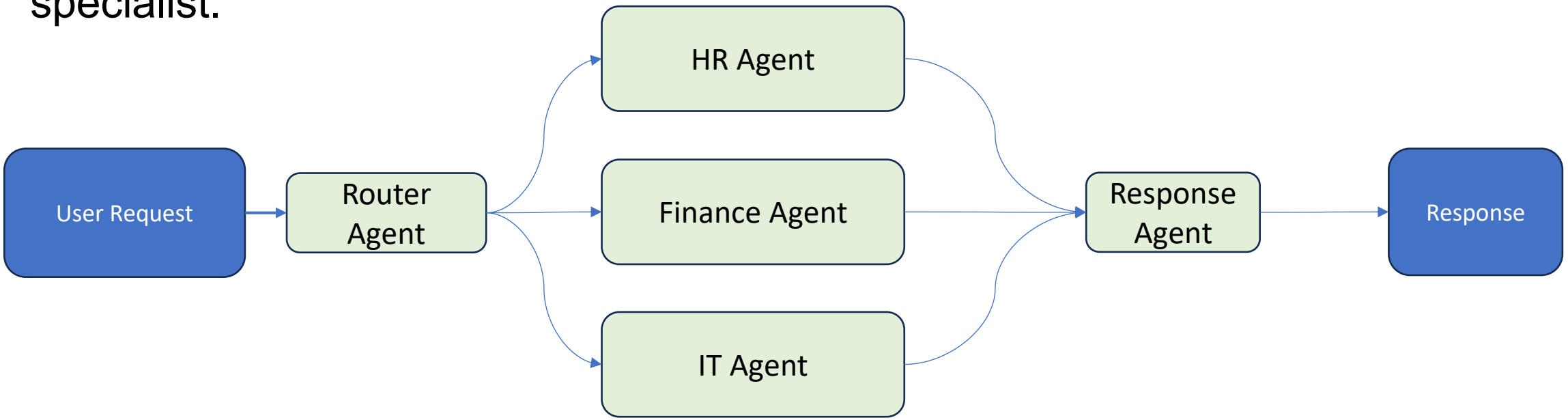
- **Use this pattern when:**
  - Subtasks are independent.
  - Several perspectives or data sources are needed.
  - Tasks can run concurrently.
  - Latency can be reduced through parallel execution.

Aggregation may use:

- Deterministic business rules;
- Structured comparison;
- Another aggregation agent;
- Human judgment.

Router–Specialist Pattern

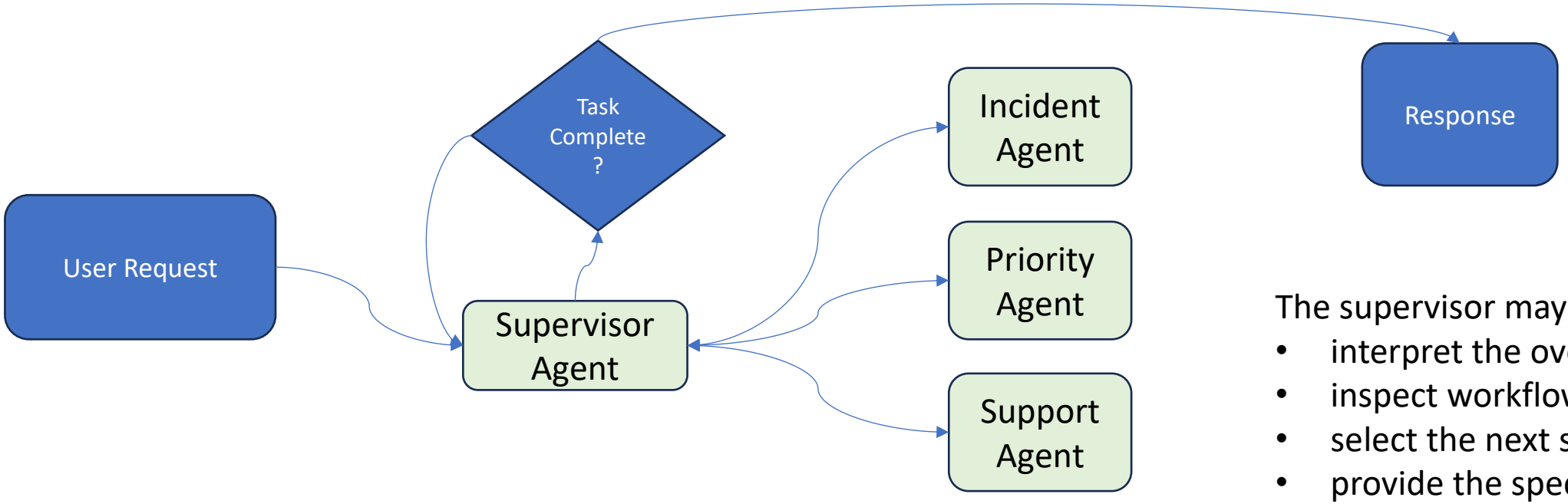
- A **router** examines the request and chooses the most appropriate specialist.



- **Use this pattern when:**
  - Requests naturally belong to distinct categories.
  - Only one specialist usually needs to handle each request.
  - Different agents require different knowledge bases or tools.

Supervisor–Specialist Pattern

- A central **supervisor agent** coordinates several specialist agents.



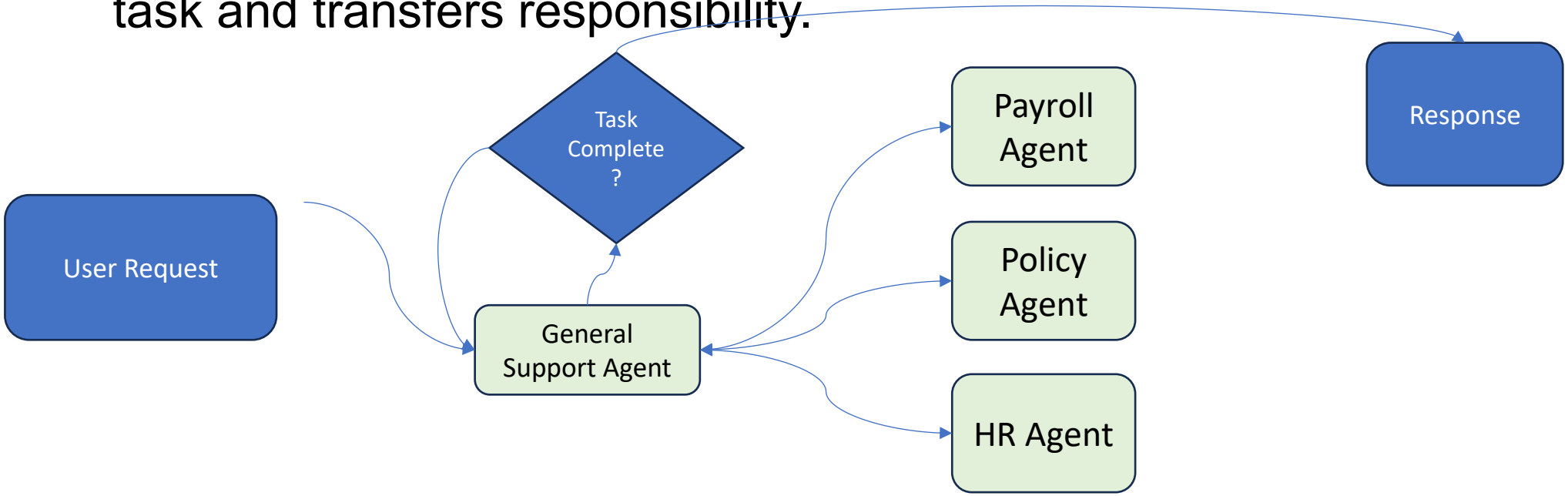
The supervisor may:

- interpret the overall task;
- inspect workflow state;
- select the next specialist;
- provide the specialist with relevant context;
- validate returned outputs;
- determine whether more work is required;
- stop or escalate the workflow.

- **Use this pattern when:**
  - Several specialists may be needed for one request.
  - The execution order cannot always be predetermined.
  - The next step depends on previous results.
  - Workflow state must be maintained.

Delegation / Handoff Pattern

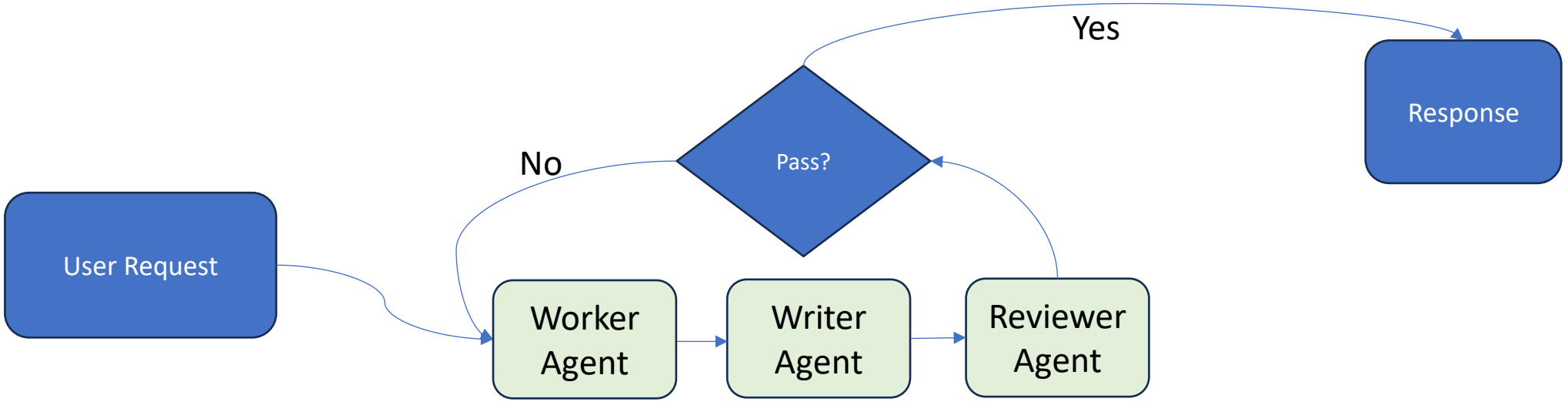
- An agent recognizes that another agent is better suited to continue the task and transfers responsibility.



- **Use this pattern when:**
  - Expertise requirements change during a workflow.
  - The next specialist cannot always be determined initially.
  - Agents can clearly identify the limits of their own responsibility.

Worker–Reviewer Pattern

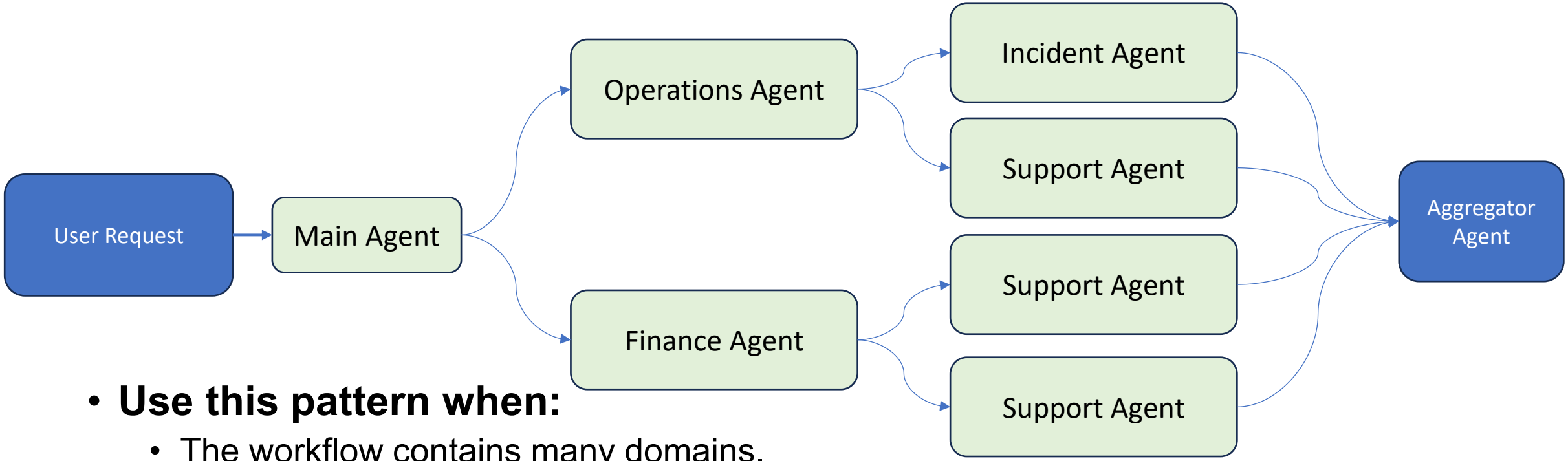
- One agent produces a result and another independently evaluates it.



- **Use this pattern when:**
  - Outputs need quality checks.
  - Policy compliance is important.
  - Hallucinations or unsupported claims must be detected.
  - High-quality drafting is required.

Hierarchical Multi-Agent Architecture

- Large systems may contain multiple levels of coordination.



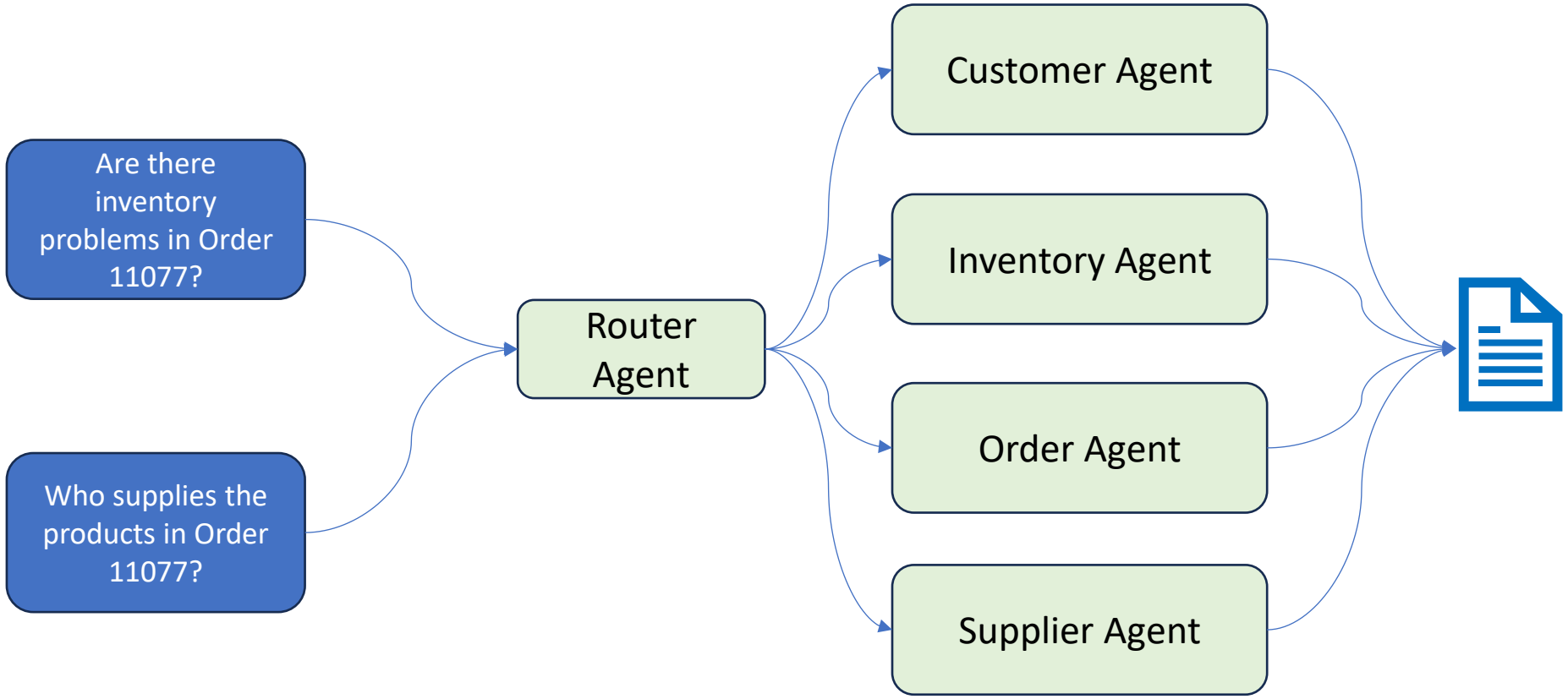
- **Use this pattern when:**
  - The workflow contains many domains.
  - Different teams require different tools and policies.
  - One supervisor would otherwise manage too many agents.
  - Different business units require separate authority boundaries.

Multi-Agent Architecture Patterns

- **Exercise 3: Implement a Router–Specialist Architecture**
- **Objectives**
  - Build a Router–Specialist workflow using LangGraph conditional edges.
  - Classify Northwind questions into order, customer, inventory, or supplier.
  - Route each request only to the specialist with the required database access.
  - Use database results as factual evidence for the specialist response.
  - Compare multiple runs and observe different agents being executed.
- **Business Case**
  - "Are there inventory problems in Order 11077?"
    - Inventory Specialist
  - "Who supplies the products in Order 11077?"
    - Supplier Specialist

Multi-Agent Architecture Patterns

- **Exercise 3: Implement a Router–Specialist Architecture**



Multi-Agent Architecture Patterns

- **Exercise 3: Implement a Router–Specialist Architecture**
-

```
show_graph(graph)
#%% EXERCISE 3: ROUTER-SPECIALIST ARCHITECTURE



# ============================================================
# ADD SUPPLIER ROLE AND DATABASE HELPER
# ============================================================

role_specs["supplier_agent"]={
    "goal":"Identify suppliers of products in the order.",
    "inputs":["order_id"],
    "outputs":["response"],
    "database":["Order Details","Products","Suppliers"]
}
```

```
============================================================
RUN 1 - INVENTORY QUESTION
============================================================
Request: Are there inventory problems in Order 11077?
Category: inventory
Agent: Inventory Agent
Response: Potential inventory shortages identified for 'Chang' (ProductID:
2), 'Aniseed Syrup' (ProductID: 3), 'Mascarpone Fabioli' (ProductID: 32), and
'Wimmers gute Semmelknödel' (ProductID: 64). 'Louisiana Hot Spiced Okra'
(ProductID: 66) is also flagged as low stock but has high replenishment in
order. Current stock levels for these items are below their respective
reorder levels.
Trace: Router -> inventory -> Inventory Agent

============================================================
RUN 2 - SUPPLIER QUESTION
============================================================
Request: Who supplies the products in Order 11077?
Category: supplier
Agent: Supplier Agent
```

Outline for this Session

- Introduction to Multi-Agent Systems
- Designing Agent Roles and Responsibilities
- Multi-Agent Architecture Patterns
- **Agent Communication, Handoffs, and Shared State**
- Stateful Multi-Agent Workflows
- Supervisor, Routing, and Workflow Control
- Human-in-the-Loop Multi-Agent Systems
- Project Case Study

How Do Agents Communicate?

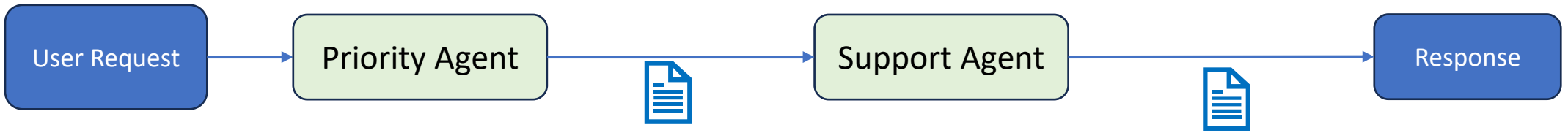
- In a multi-agent system, agents usually do not need unrestricted conversations with each other.
- Agents communicate by exchanging:
  - structured inputs;
  - structured outputs;
  - workflow state;
  - retrieved evidence;
  - task status;
  - handoff information;
  - errors or escalation signals.

Three Ways Information Moves Between Agents

- Direct Message Passing
- Handoff
- Shared State

Direct Message Passing

- **Direct Message Passing** occurs when one agent sends its output directly to another agent that needs it for the next task.

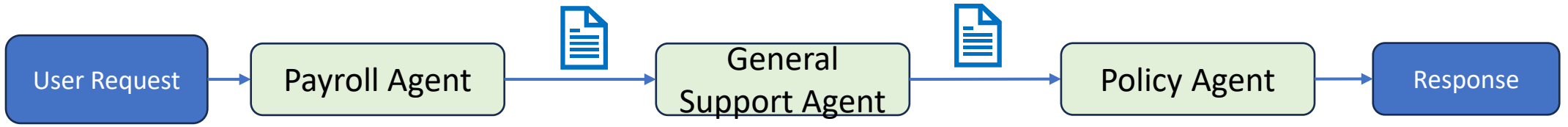


- Agent A completes a task and produces a result.
- The result is passed directly to Agent B.
- Agent B uses that message as input for its own task.

What Is an Agent Handoff?

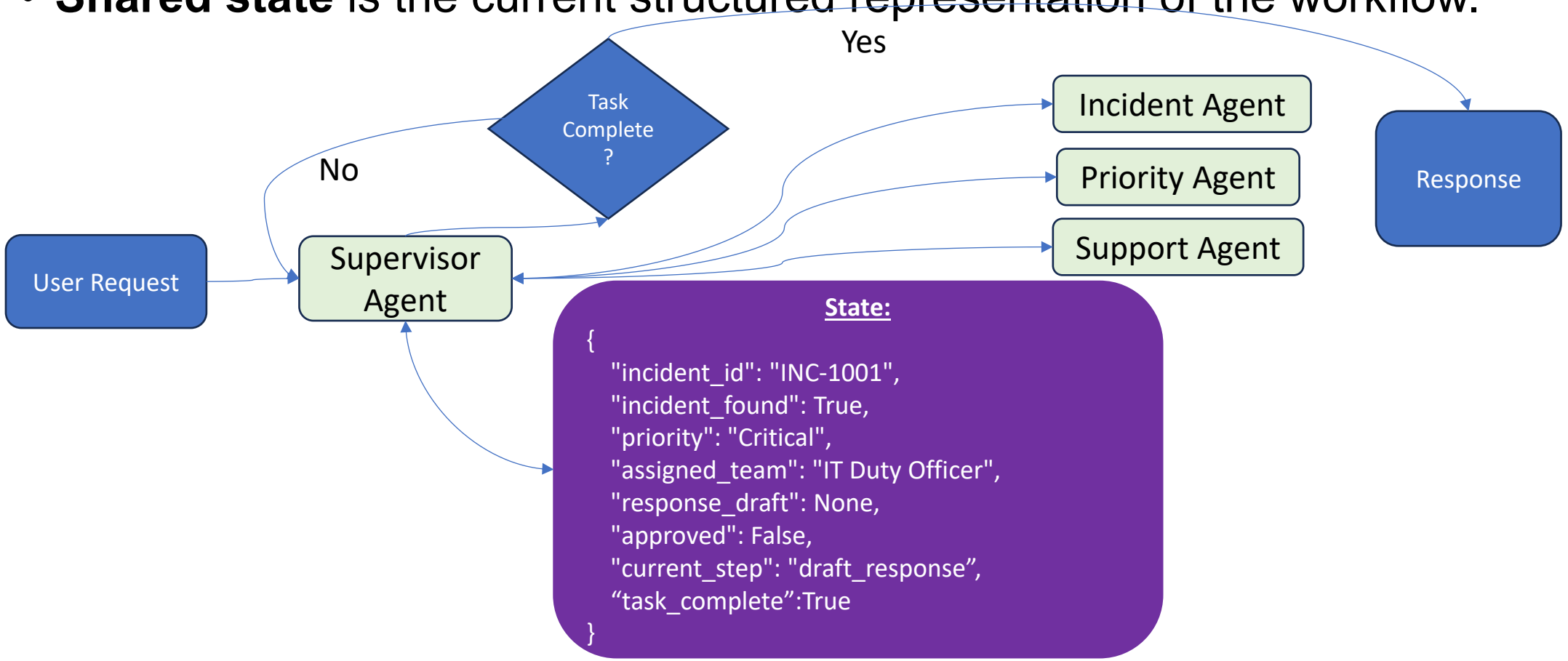
- A **handoff** occurs when one agent transfers responsibility for part of the workflow to another agent.
- A good handoff should explain:
  - what the current task is;
  - what has already been completed;
  - what evidence has been collected;
  - why the handoff is occurring;
  - what the receiving agent should do next.
- Direct message passing transfers information. A handoff transfers responsibility.

```
{
  "from_agent": "incident_agent",
  "to_agent": "priority_agent",
  "task": "Assess incident priority",
  "incident_id": "INC-1001",
  "evidence": {
    "system": "Payroll",
    "status": "Unavailable",
    "affected_users": "All employees"
  },
  "required_output": "Priority
assessment",
  "reason_for_handoff": "Incident
identified"
}
```



Shared States

- **Shared state** is the current structured representation of the workflow.



- Shared state preserves information for the wider workflow.

State Changes Across a Multi-Agent Workflow

Initial state

```
incident_id = None
priority = None
assigned_team = None
response_draft = None
approved = False
```

After Incident Agent

```
incident_id =
INC-1001
incident_found =
True
```

After Priority Agent

```
priority = Critical
```

After Support Agent

```
assigned_team = IT Duty
Officer
```

After Response Agent

```
response_draft =
"..."
```

After Human Review

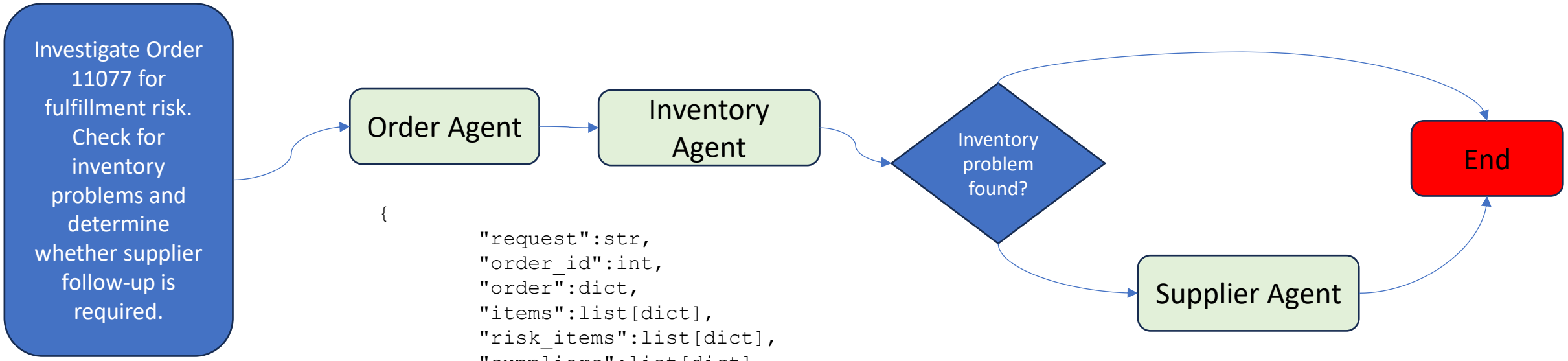
```
approved =
True
```

Communication, Handoffs, and Shared State

- **Exercise 4: Transfer Context and Responsibility**
- Objectives
  - An operations manager asks the system to investigate **Order 11077** for fulfillment risk.
  - The **Order Agent** retrieves the order and transfers the investigation to the Inventory Agent.
  - The **Inventory Agent** checks ordered products for low-stock or potential-shortage conditions.
  - If inventory risk is found, the Inventory Agent creates a structured handoff to the **Supplier Agent**.

Communication, Handoffs, and Shared State

- **Exercise 4: Transfer Context and Responsibility**




```
{
        "request":str,
        "order_id":int,
        "order":dict,
        "items":list[dict],
        "risk_items":list[dict],
        "suppliers":list[dict],
        "handoff":dict,
        "supplier_review_required":bool,
        "response":str,
        "status":str,
        "status_history":list[str],
        "memory":list[str],
        "trace":list[str]
},
```

Communication, Handoffs, and Shared State

- **Exercise 4: Transfer Context and Responsibility**

```
show_graph(graph)
#%% EXERCISE 4: AGENT COMMUNICATION, HANDOFFS, AND SHARED STATE
# ============================================================
# NEW REUSABLE HELPERS
# ============================================================

def set_status(state,status,agent):
    old=state.get("status","START")
    return {
        "status":status,
        "status_history":state.get("status_history",[])+[
            f"{old} -> {status} ({agent})"
        ]
    }

def add_memory(state,message):
    return state.get("memory",[])+[message]

def make_handoff(from_agent,to_agent,task,evidence):
    return {
```

```
Training/1. LLM Engineering/Python Code/Session 06.py

REQUEST
Investigate Order 11077 for fulfillment risk. Check for inventory probl
and determine whether supplier follow-up is required.

FINAL HANDOFF
from: Inventory Agent
to: Supplier Agent
task: Review suppliers for inventory-risk products.
evidence: {'order_id': 11077, 'risk_products': [{'ProductID': 2,
'ProductName': 'Chang', 'InventoryStatus': 'Potential shortage'},
{'ProductID': 3, 'ProductName': 'Aniseed Syrup', 'InventoryStatus': 'L
```

Outline for this Session

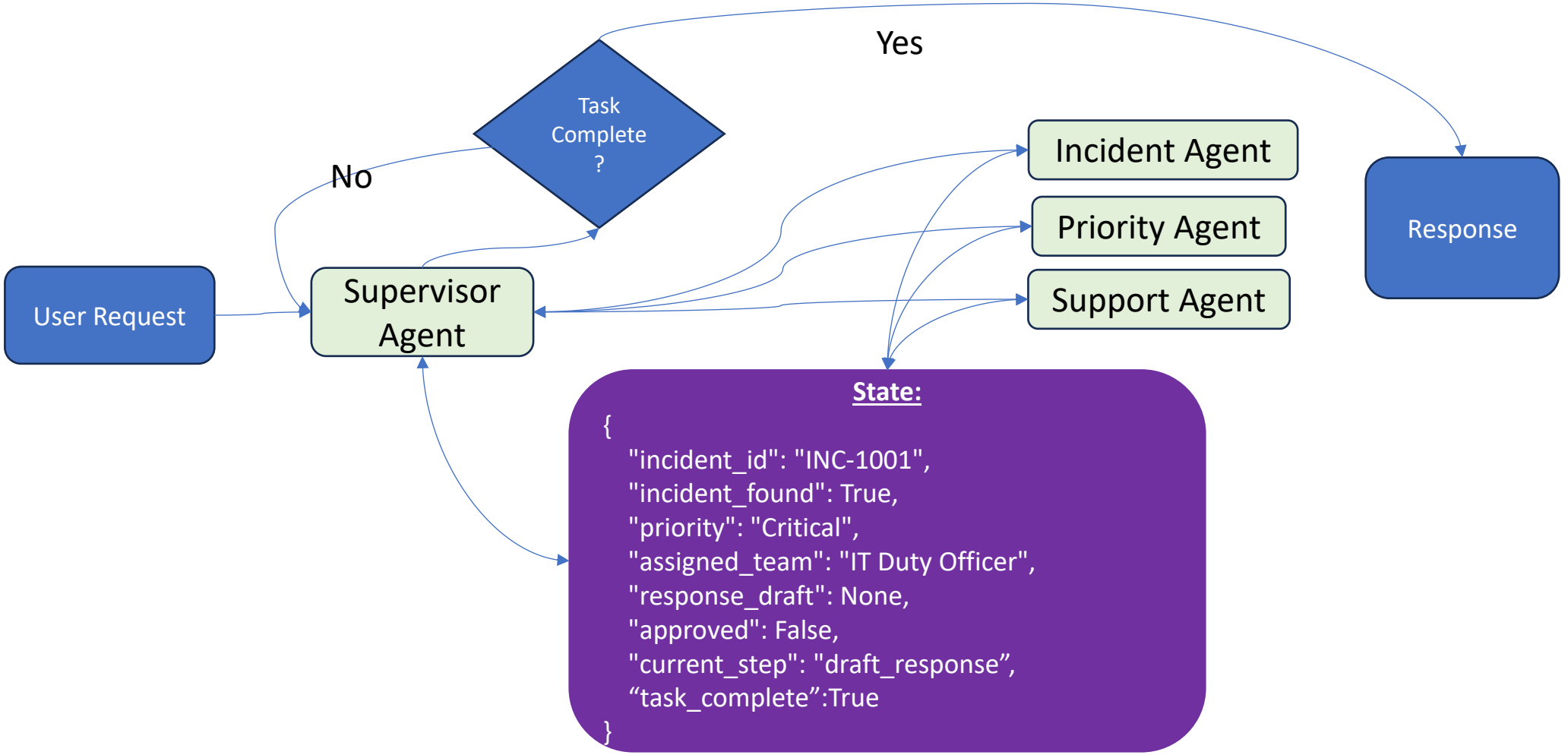
- Introduction to Multi-Agent Systems
- Designing Agent Roles and Responsibilities
- Multi-Agent Architecture Patterns
- Agent Communication, Handoffs, and Shared State
- **Stateful Multi-Agent Workflows**
- Supervisor, Routing, and Workflow Control
- Human-in-the-Loop Multi-Agent Systems
- Project Case Study

Stateful Multi-Agent Workflow

- A **stateful multi-agent workflow** is a multi-agent process that preserves information about the current execution as work moves from one agent to another.
  - The workflow remembers what has already happened.
  - Agents can read relevant information from the current state.
  - Agents can update the state after completing their tasks.
  - The next step can depend on the current state.
  - The workflow can pause and later resume from the same point.
  - Errors and retries can be handled without restarting the entire process.

Why Does a Multi-Agent System Need State?

- Without state, every agent may need to reconstruct the workflow context.



Anatomy of Workflow State

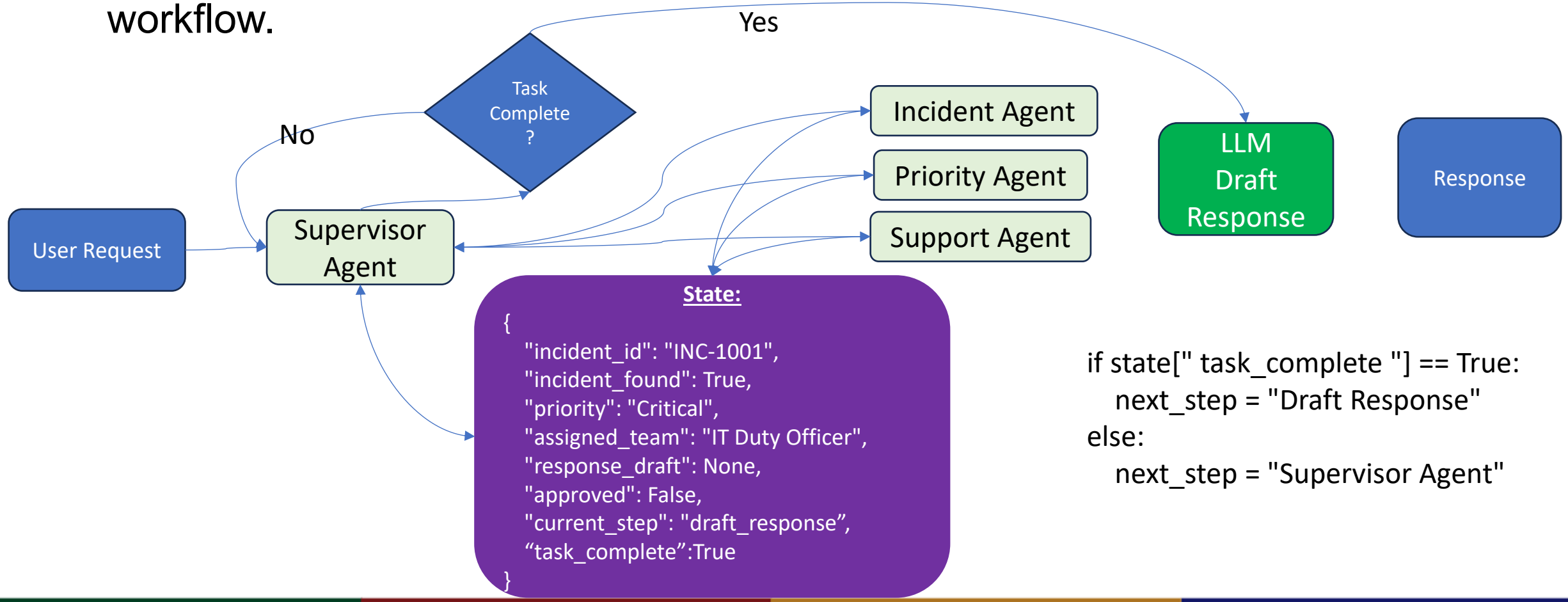
- Workflow state normally contains both **business data** and **workflow-control data**.
- Business state describes the problem. Workflow state describes the progress of the process.

```
state = {
    "request": "Payroll system unavailable",
    "incident_id": "INC-1001",
    "priority": "Critical",
    "assigned_team": "IT Duty Officer",
    "response_draft": None,
    "current_step": "draft_response",
    "status": "in_progress",
    "approved": False,
    "retry_count": 0,
    "error": None
}
```

Business State	Workflow State
Incident ID	Current step
Priority	Workflow status
Assigned team	Retry count
Retrieved evidence	Approval status
Response draft	Error information

Conditional Transitions

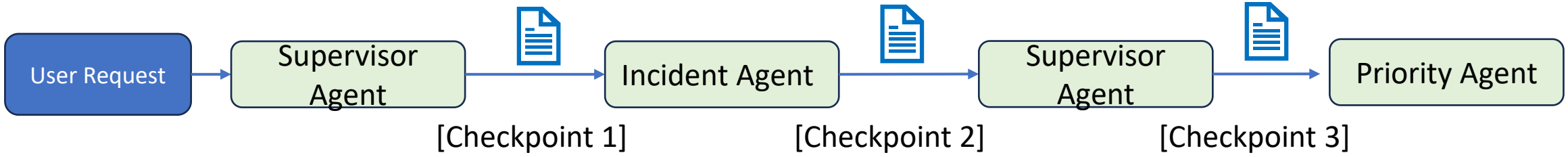
- The next step does not always have to be predetermined.
- Conditional routing converts a fixed agent sequence into an adaptive workflow.




```
if state[" task_complete "] == True:
    next_step = "Draft Response"
else:
    next_step = "Supervisor Agent"
```

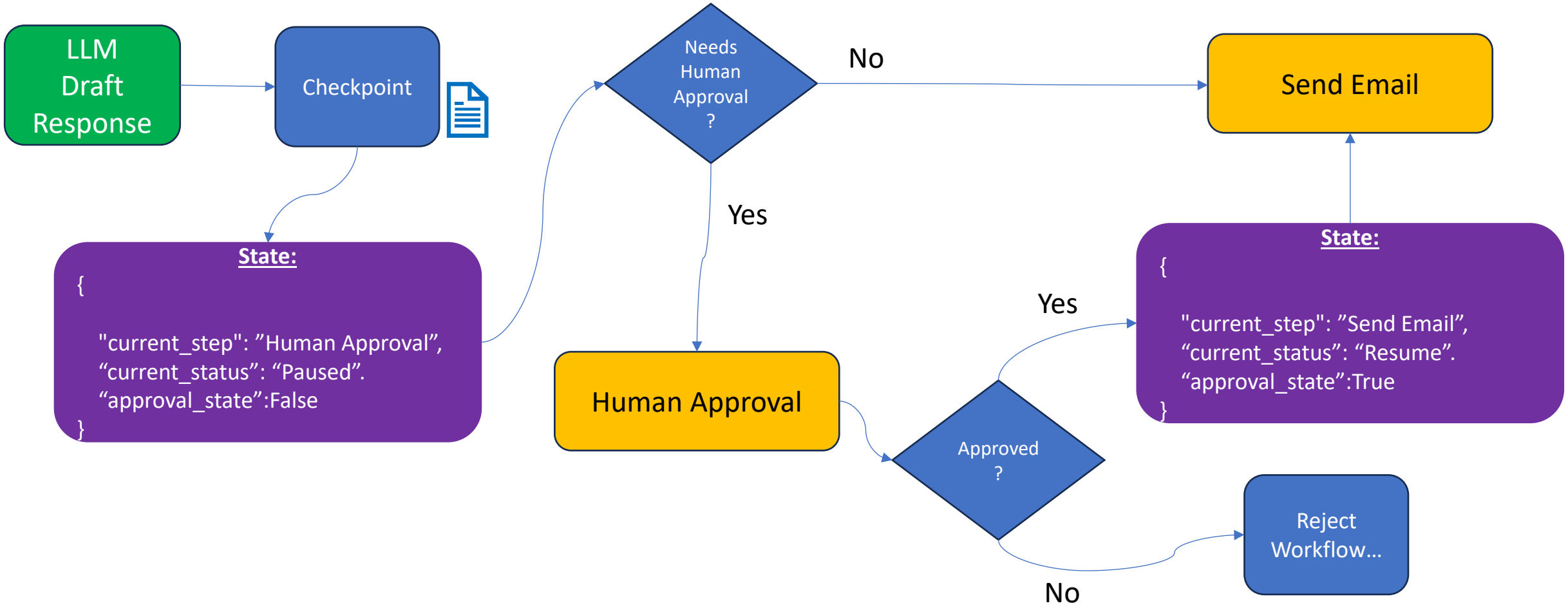
Checkpoints and Persistence

- **Definition:** A **checkpoint** saves the workflow state at a particular point in execution.
- If the workflow matters beyond one immediate model call, its state should usually be persisted.
- Checkpoints allow the system to preserve:
  - current workflow position;
  - agent outputs;
  - retrieved evidence;
  - approval status;
  - retry information;
  - errors and execution metadata.



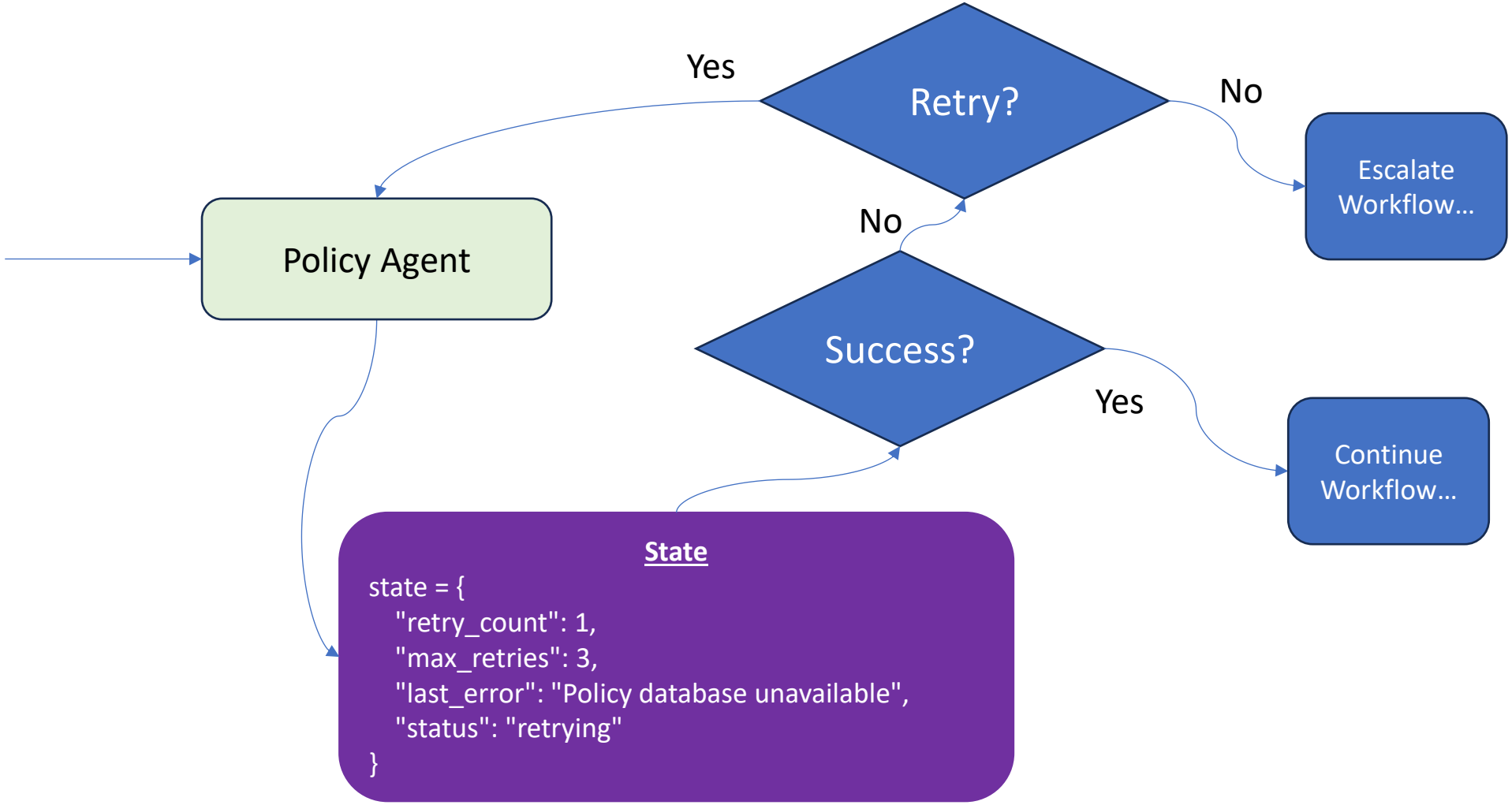
Pause and Resume

- State persistence allows workflows to pause without losing progress.



Retry, Recovery, and Failure Handling

- State also supports controlled recovery from failures.



A Stateful Controlled Workflow

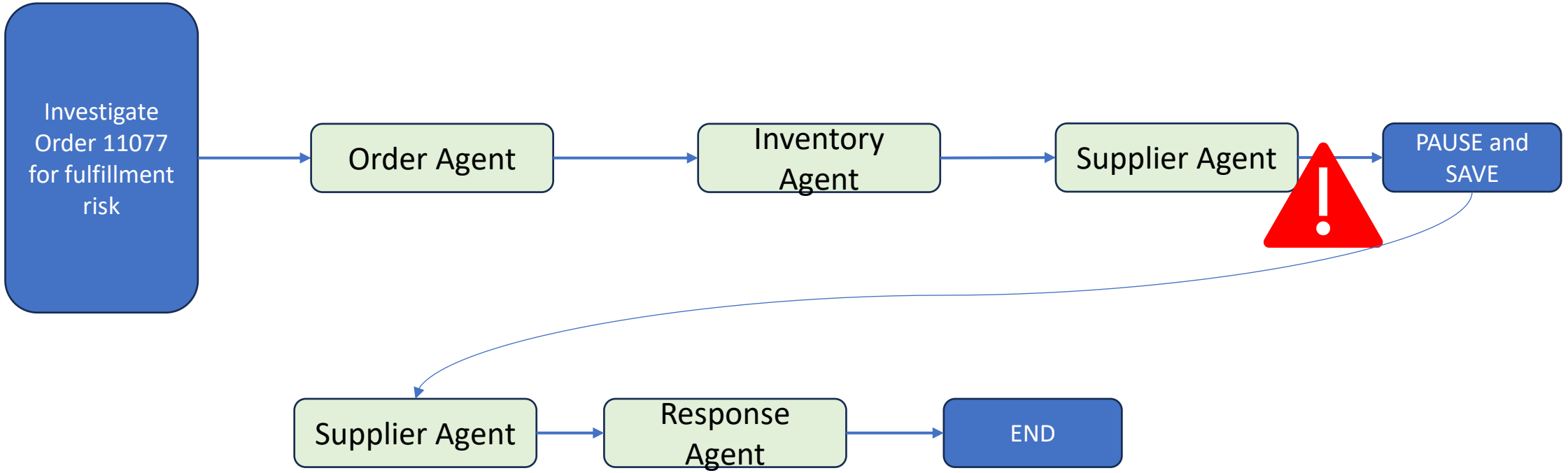
- The workflow includes:
  - **agents** that perform specialized tasks;
  - **state** that carries information;
  - **nodes** that represent workflow steps;
  - **edges** that connect those steps;
  - **conditions** that control routing;
  - **checkpoints** that persist progress;
  - **retries** for controlled recovery;
  - **human approval** for consequential actions.
- A robust multi-agent system should know not only what the agents know, but also where the workflow is, what has already happened, and what is allowed to happen next.

Stateful Multi-Agent Workflows

- **Exercise 5: Pause, Save, and Resume a Workflow**
- **Objectives**
  - Add checkpointing to a Northwind multi-agent workflow.
  - Preserve shared state and workflow memory after completed agent actions.
  - Pause an order investigation before the next specialist executes.
  - Inspect the saved checkpoint, including the next node to execute.
  - Resume the same workflow using its thread_id.
  - Verify that completed agents do not need to restart the investigation.
- **Business Case**
  - Northwind is investigating **Order 11077** for fulfillment risk. The Order Agent retrieves the order and the Inventory Agent identifies products requiring supplier review.
  - The investigation must temporarily stop before supplier follow-up—for example, because the operations team continues the case later.

Stateful Multi-Agent Workflows

- **Exercise 5: Pause, Save, and Resume a Workflow**



Stateful Multi-Agent Workflows

- **Exercise 5: Pause, Save, and Resume a Workflow**

```python
#%% EXERCISE 5: STATEFUL FAILURE AND RECOVERY


from langgraph.checkpoint.memory import InMemorySaver

# ============================================================
# SIMULATE SUPPLIER LLM AVAILABILITY
# ============================================================

SUPPLIER_LLM_AVAILABLE=False

# ============================================================
# SUPPLIER AGENT WITH SIMULATED LLM FAILURE
# ============================================================

def supplier_agent_recovery(state):
    print("Supplier Agent started.")

    if not SUPPLIER_LLM_AVAILABLE:
```

```
============================================================
PHASE 1 - SUPPLIER LLM UNAVAILABLE
============================================================
Supplier Agent started.
Supplier LLM unreachable.

WORKFLOW FAILED
Supplier Agent LLM is unavailable.

SAVED STATE
Status: SUPPLIER REVIEW REQUIRED
Next node: ('supplier_agent',)
Trace: Order Agent -> Inventory Agent

============================================================
PHASE 2 - SUPPLIER LLM RESTORED
```

Outline for this Session

- Introduction to Multi-Agent Systems
- Designing Agent Roles and Responsibilities
- Multi-Agent Architecture Patterns
- Agent Communication, Handoffs, and Shared State
- Stateful Multi-Agent Workflows
- **Supervisor, Routing, and Workflow Control**
- Human-in-the-Loop Multi-Agent Systems
- Project Case Study

What Is the Supervisor?

- A **supervisor** coordinates the execution of a multi-agent workflow.
- The supervisor may:
  - inspect the current workflow state;
  - determine which agent should act next;
  - provide the selected agent with relevant context;
  - validate the returned result;
  - update workflow progress;
  - decide whether to continue, retry, escalate, or stop.

Router vs Supervisor vs Workflow Controller

Component	Main Responsibility
Router	Selects which agent or path should handle a request.
Supervisor	Coordinates multiple agents across the workflow.
Workflow Controller	Enforces transitions, rules, validation, retries, and stop conditions.

What Does the Supervisor Use to Decide?

- The supervisor should not make decisions from the user request alone.
- It should consider:
  - the user goal;
  - current workflow state;
  - completed tasks;
  - previous agent outputs;
  - confidence or validation results;
  - available agents;
  - allowed transitions;
  - errors or retry counts;

Routing Strategies

- 1. Rule-Based Routing
  - Best when:
    - rules are known;
    - decisions must be predictable;
    - compliance is important.
- 2. LLM-Based Routing
  - Best when:
    - requests are ambiguous;
    - routing requires interpretation.

App

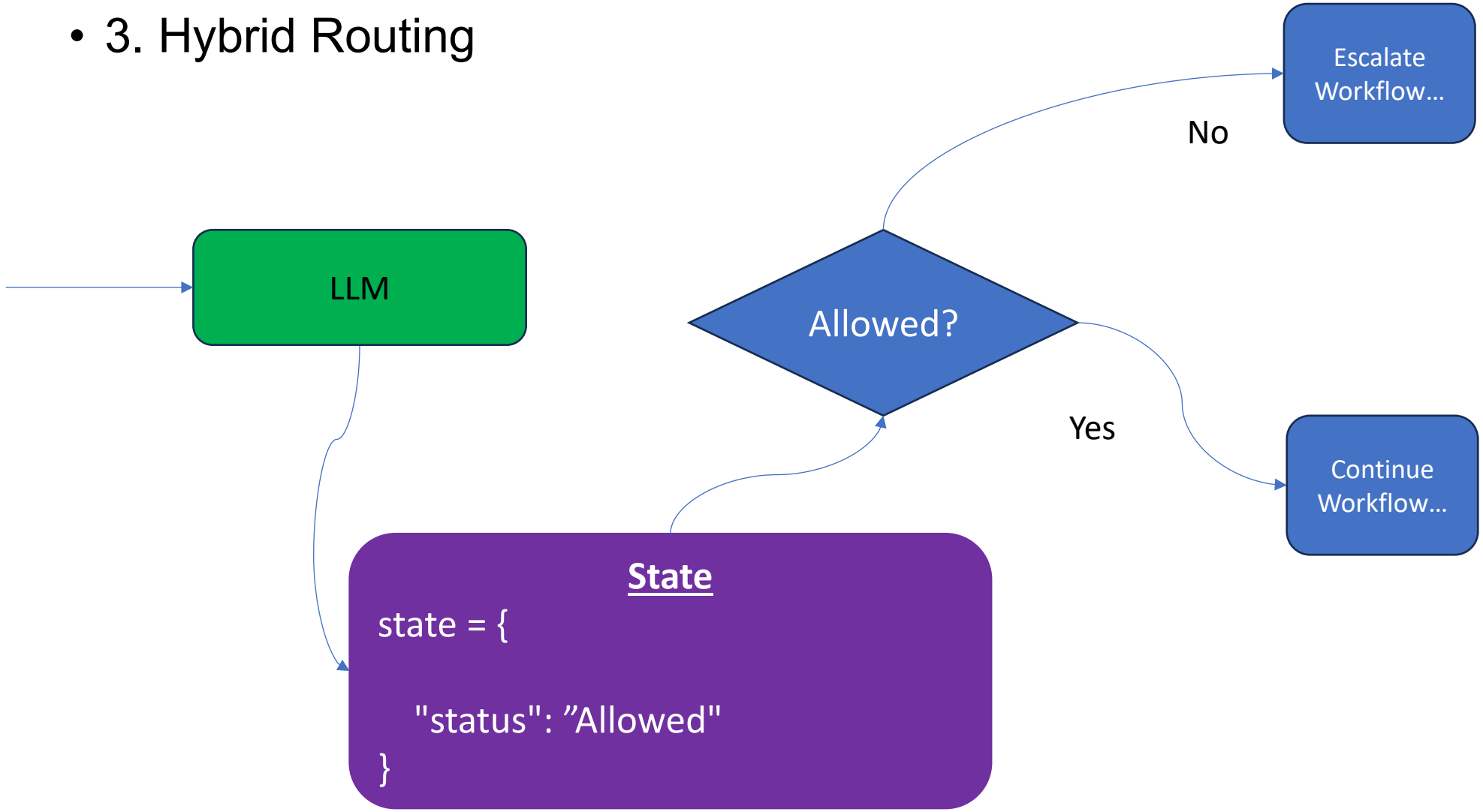
```
if state["priority"] == "Critical":
    next_step = "human_review"
else:
    next_step = "response_agent"
```

LLM

```
"Select the next agent:
incident_agent
priority_agent
support_agent
response_agent"
```

Routing Strategies

- 3. Hybrid Routing



Controlled Routing: Allowed Transitions

- An agent should not be able to jump directly to any other agent or action.

```
allowed_transitions = {
    "incident_agent": ["priority_agent", "escalate"],
    "priority_agent": ["support_agent", "human_review"],
    "support_agent": ["response_agent"],
    "response_agent": ["human_approval"],
    "human_approval": ["execute", "revise"]
}
```

Stop Conditions, Retries, and Escalation

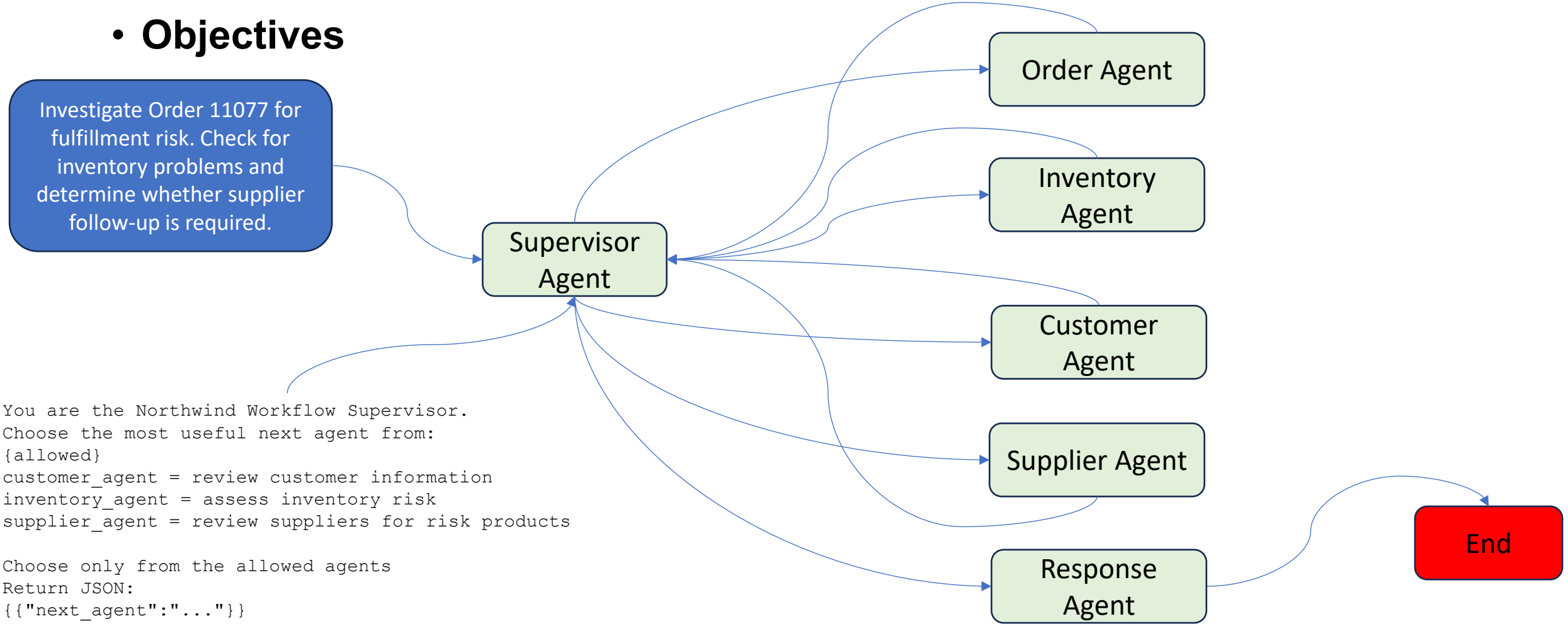
- A controlled workflow must know when **not to continue**.
- Possible stop conditions include:
  - task completed;
  - required information unavailable;
  - maximum retries reached;
  - confidence below acceptable threshold;
  - invalid or conflicting outputs;
  - human rejection;
  - unauthorized action requested.

Supervisor, Routing, and Workflow Control

- **Exercise 6: Let the LLM Propose; Let the Application Control**
- **Objectives**
  - An operations manager asks the system to investigate **Order 11077** and recommend the appropriate operational action.
  - The investigation may require order, customer, inventory, and supplier information.
  - Not every specialist should run automatically or in a fixed sequence.
  - A **Supervisor Agent** decides which specialist should act next based on the current workflow state.
  - The Response Agent prepares the final operational assessment after all required reviews are complete.

Supervisor, Routing, and Workflow Control

- **Exercise 6: Let the LLM Propose; Let the Application Control**
- **Objectives**



Supervisor, Routing, and Workflow Control

- **Exercise 6: Let the LLM Propose; Let the Application Control**

```python
#%% EXERCISE 6: SUPERVISOR, ROUTING, AND WORKFLOW CONTROL
# Run Exercises 1-5 first

# ============================================================
# EXTEND PREVIOUS STATE
# ============================================================

SupervisorState=TypedDict(
    "SupervisorState",
    {
        **HandoffState.__annotations__,
        "customer":dict,
        "customer_summary":str,
        "allowed_actions":list[str],
        "next_agent":str
    },
    total=False
)
```

```
SUPERVISOR TRACE
- Supervisor -> order_agent [allowed: order_agent]
- Order Agent
- Supervisor -> customer_agent [allowed: customer_agent, inventory_a
- Customer Agent
- Supervisor -> inventory_agent [allowed: inventory_agent]
- Inventory Agent
- Supervisor -> supplier_agent [allowed: supplier_agent]
- Supplier Agent
- Supervisor -> response_agent [allowed: response_agent]
- Response Agent
```

Outline for this Session

- Introduction to Multi-Agent Systems
- Designing Agent Roles and Responsibilities
- Multi-Agent Architecture Patterns
- Agent Communication, Handoffs, and Shared State
- Stateful Multi-Agent Workflows
- Supervisor, Routing, and Workflow Control
- **Human-in-the-Loop Multi-Agent Systems**
- Project Case Study

What Is Human-in-the-Loop?

- **Human-in-the-Loop (HITL)** means that selected points in an agentic workflow require human input before the workflow can continue.
- The human may:
  - review an agent's recommendation;
  - approve or reject a proposed action;
  - correct an agent's output;
  - provide missing information;
  - resolve conflicting agent recommendations;
  - override a proposed decision;
- The goal of HITL is not to keep humans involved everywhere. It is to place human judgment and authority exactly where automated decisions become uncertain, exceptional, or consequential.

The Rationale for HITL

- Multiple agents can increase capability—but they can also increase uncertainty and coordination risk.
- Potential issues include:
  - an agent may misinterpret the user's intent;
  - retrieved evidence may be incomplete;
  - agents may produce conflicting conclusions;
  - a supervisor may select the wrong workflow path;
  - an agent may propose an unauthorized action;
  - confidence may be too low for automatic execution;
  - an apparently valid output may have significant business consequences.
- The more consequential the action, the stronger the case for human oversight.

Where Should Humans Enter the Workflow?

- Human intervention should be placed at **control points where risk or uncertainty becomes significant**.

Workflow Situation	Appropriate Human Role
Low-confidence classification	Confirm or correct
Missing evidence	Provide information
Conflicting agents	Resolve disagreement
Policy exception	Make judgment
High-priority incident	Confirm escalation
External communication	Review message
Financial transaction	Authorize action
Irreversible operation	Final approval

Common HITL Intervention Patterns

- 1. Approve / Reject
- 2. Review and Edit
- 3. Clarification
- 4. Escalation
- 5. Exception Handling

Agent Proposal
↓
Human Review
↓
Approve or Reject

Agent Draft
↓
Human Edit
↓
Continue

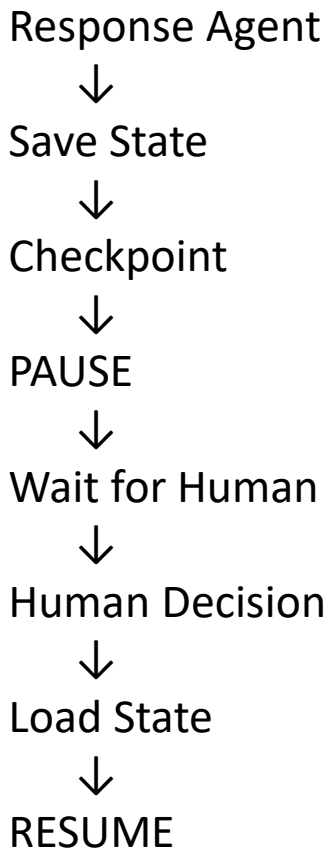
Missing Information
↓
Ask Human
↓
Human Supplies Data
↓
Resume

Agent Cannot Resolve
↓
Human Expert
↓
Decision

Normal Workflow
↓
Rule Exception
↓
Human Decision

HITL Requires State and Persistence

- Human review may not happen immediately.
- A person could respond:
  - seconds later;
  - several minutes later;
  - hours later;
  - the next business day.
- Therefore, the workflow must preserve its state.
- The human interaction itself should become part of the workflow record.

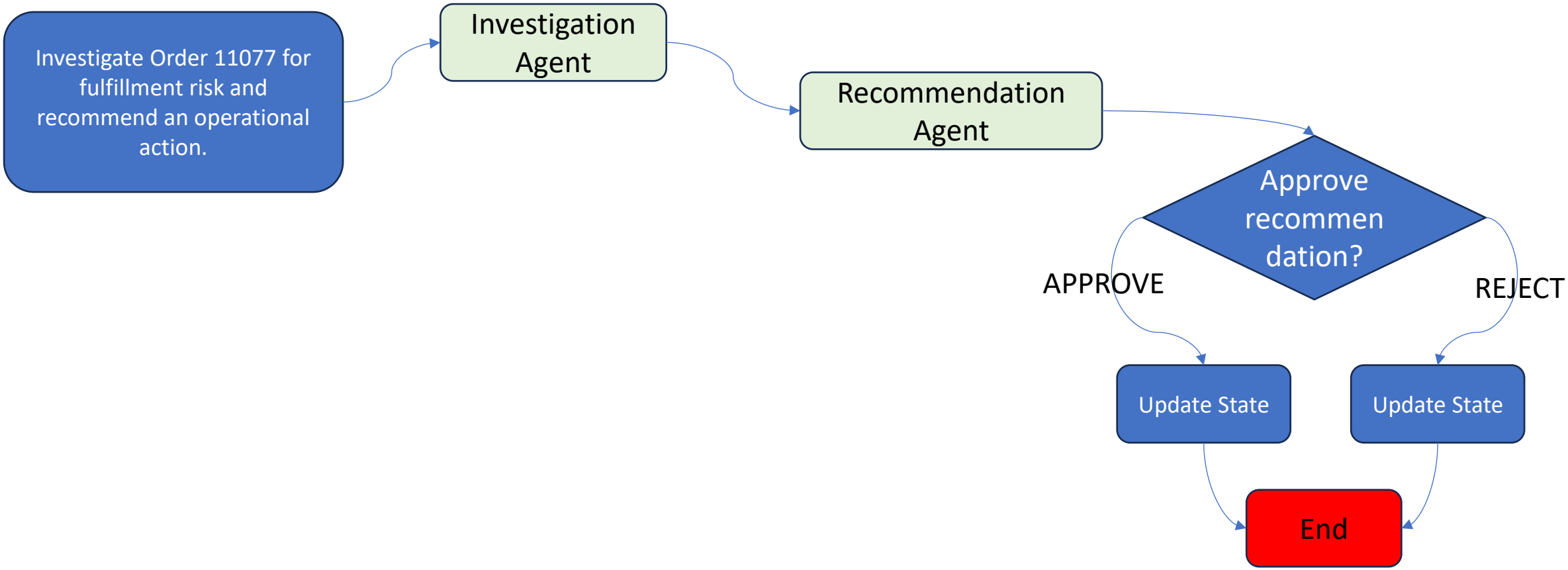


HITL

- **Exercise 7: Human Approvers**
- **Objectives**
  - An operations manager asks the system to investigate **Order 11077** and recommend the appropriate action.
  - The **Investigation Agent** retrieves the order, customer, and inventory facts from the database.
  - The system determines the level of fulfillment risk based on identified inventory issues.
  - The **Recommendation Agent** proposes one action: **PRIORITIZE, HOLD, or STANDARD**.
  - Before execution, the workflow pauses and sends the recommendation to a **Human Reviewer**.
  - The reviewer can **approve** or **reject** the recommendation.

HITL

- **Exercise 7: Human Approvers**



HITL

- **Exercise 7: Human Approvers**

```
#%% EXERCISE 7: HUMAN-IN-THE-LOOP APPROVAL


from langgraph.checkpoint.memory import InMemorySaver
from langgraph.types import interrupt,Command

# ============================================================
# STATE + SMALL NEW HELPERS
# ============================================================

HITLState=TypedDict(
    "HITLState",
    {
        **HandoffState.__annotations__,
        "customer":dict,
        "risk_level":str,
        "recommendation":str,
        "recommendation_reason":str,
        "approval_status":str
    },
    total=False
)

def show_approval(data):
    print("\nHUMAN APPROVAL REQUIRED")
    print("Order:",data["order_id"])
    print("Customer:",data["customer"])
    print("Risk:",data["risk_level"])
    print("Risk items:",data["risk_count"])
    print("AI recommendation:",data["recommendation"])
    print("Reason:",data["reason"])
```

```
HUMAN APPROVAL REQUIRED
Order: 11077
Customer: Rattlesnake Canyon Grocery
Risk: HIGH
Risk items: 5
AI recommendation: HOLD
Reason: Multiple high-risk items in this order, particularly Chang
(potential shortage) and low-stock critical components like Louisian
Spiced Okra and Wimmers gute Semmelknödel, require immediate invento
review to prevent stockouts before fulfillment.

Approve recommendation? [approve/reject]: approve

FINAL DECISION
AI Recommendation: HOLD
Human Decision: APPROVE
Final Status: COMPLETED
```

Outline for this Session

- Introduction to Multi-Agent Systems
- Designing Agent Roles and Responsibilities
- Multi-Agent Architecture Patterns
- Agent Communication, Handoffs, and Shared State
- Stateful Multi-Agent Workflows
- Supervisor, Routing, and Workflow Control
- Human-in-the-Loop Multi-Agent Systems
- Project Case Study

Case 7

- **Business Case:** Build a complete multi-agent application that investigates an order exception, coordinates specialist agents, survives failures, and requires human approval before action.
  - A **Supervisor Agent** decides which specialist should act next
  - **Order, Customer, Inventory, and Supplier Agents** retrieve facts and reason only within their assigned roles.
  - Agents share evidence through a common workflow state and structured handoffs.
  - The application limits the Supervisor to a safe set of allowed_actions.
  - A simulated **Supplier Agent LLM outage** causes the workflow to fail
  - LangGraph checkpointing preserves the transaction so the same thread_id can resume without restarting completed agents.
  - Once the Supplier LLM is restored, the workflow continues from the failed step.
  - The **Response Agent** synthesizes the investigation and recommends PRIORITIZE, HOLD, or STANDARD.
  - The workflow pauses for **human approval**
  - Memory, status history, handoffs, recommendations, human decisions, and execution trace provide end-to-end auditability.

Project Capstone

- Capstone Project Work: Extend Your App into a Multi-Agent System
  - Identify at least **three distinct agent roles** with clear responsibilities and non-overlapping tasks.
  - Define what data, tools, or functions each agent is allowed to use.
  - Introduce a **Supervisor or Router Agent** that decides which specialist should act next.
  - Use a shared workflow state to pass relevant facts, intermediate results, status, and trace information between agents.
  - Add a **Human-in-the-Loop approval point** before one consequential action is executed.
  - Record the final recommendation, human decision, workflow status, memory, and execution trace

Outline for this Session

- Introduction to Multi-Agent Systems
- Designing Agent Roles and Responsibilities
- Multi-Agent Architecture Patterns
- Agent Communication, Handoffs, and Shared State
- Stateful Multi-Agent Workflows
- Supervisor, Routing, and Workflow Control
- Human-in-the-Loop Multi-Agent Systems
- Project Case Study